Stream Decoding Step by Step — A Tutorial

This is the third, most detailed version of the guide on decoding a DataShield stream — "for technical-college students": every term is introduced from scratch, every pipeline step is walked through with numbers and traces. The formal monograph is `DecoderGuide.Academic.en.md`; the short engineering retelling is `DecoderGuide.Engineer.en.md` (their section numbering matches each other; this tutorial uses its own, lesson-style numbering). A general overview is in `AlgorithmGuide.en.md` (sections 5–9); the forward transformation is the `EncoderGuide.*` family. Assembling the file from what was received is a separate topic covered by the `AssemblyGuide.*` family; here it is only "the door we enter through".

After reading this tutorial you should be able to explain in your own words: how the decoder fishes packets out of a noisy stream, why the header is recognized on its own while a sector needs "its" header, what happens when a sector arrives before the header, and why `Complete` must be called at the end.

1. Vocabulary: the words we cannot do without

Term	What it actually is
Decoder	The `FileDecoder` class: a byte stream → accumulated reception slots (and, on request, the assembled file).
FEC stream	The input: Base64 text (`.DataShield.txt`) or raw packets (`.DataShield.bin`).
Pipeline	The chain "source → filter → scanner → accumulator", wired together by data-ready events.
Source (`IDataSource`)	Supplies bytes: from an array, a stream, or a file.
Processor (`IDataProcessor`)	Both a source and a sink: takes someone's bytes, processes them, emits its own. Chains are built this way.
Filter	`ByteRangeFilter`: passes only Base64-alphabet characters, discarding the rest.
Scanner	`SlidingWindowScanner`: a sliding window that fishes 75-byte packets out of the stream.
Window	A fixed-length slice of the stream: 100 characters (txt) or 75 bytes (bin).
Accumulator	`StreamProcessor`: collects packets, creates slots, distributes sectors to slots.
Reception slot	`ReceptionSlot`: everything received about one file: header + sector versions.
`H5`	The 24-byte header hash; the "seed" without which a sector cannot be verified.
Retained data	The entire stream the scanner has passed through itself — kept for rebinding.
Rebinding	Re-checking retained data against a newly arrived header.
Forward pass	The main forward scan, window by window.
Rescan	An exhaustive re-check of retained data at every position.

2. What the decoder does: the big picture

Input: a byte stream (text or binary), possibly damaged: with garbage, holes, splices. Output: reception slots, one per recognized file. From a slot you can then try to assemble the file (`TryAssemble` — the subject of `AssemblyGuide.Tutorial.en.md`). The pipeline:

```
byte source (array / stream / file)
        |
        ▼ (txt mode only)
  Base64 filter: keep A-Z a-z 0-9 + /, throw away the rest
        |
        ▼
  sliding-window scanner: 100 chars (txt) or 75 bytes (bin)
        |  window not recognized → shift by 1; recognized → emit packet, jump by the wi
  ndow
        ▼
  StreamProcessor accumulator: packets → reception slots
        |  header: H5 check (autonomous) → slot / copy counter
        |  sector: D3 check with the H5 seed → a version in the slot
        ▼
  slots → TryAssemble (the AssemblyGuide family)
```

3. Input formats and autodetection

Two formats, mirroring the encoder:

Format	Extension	What the decoder sees
Base64	`.DataShield.txt`	lines of 100 characters, possibly garbage and decorative framing
Binary	`.DataShield.bin`	raw 75-byte packets, possibly holes and desynchronization

When working with a file (`PacketIO.ScanFile`) the format is detected by extension: `.txt` → Base64, `.bin` → Binary, anything else → assumed Base64. When working with a stream (`Scan(Stream, format)`) the format is given explicitly — a stream has no extension.

4. The pipeline in plain terms: sources and processors

Everything in the pipeline is a "data source" with a `DataReady` event ("data is ready — take it"). A processor is a source that also has an input: `Attach(source)` subscribes to its data; `Complete()` means "the input has ended, deliver the remainder". Chains of any length are built this way.

Three practical rules:

1. **Starting is cascading.** `source.Start()` launches the whole chain; the source reads a block, fills its buffer, announces `DataReady`; the next module reads and processes until exhausted.
2. **Completion is in order.** At the end the pipeline is closed strictly from the head: `filter.Complete()`, then `scanner.Complete()`, then `processor.Complete()`. Each delivers its buffer remainder to the next.
3. **Errors travel the `Error` channel.** If the source fails (an I/O fault), its `Error` property holds the exception and its `Completion` task ends with that error; processors merely relay the error up the chain. An error is never swallowed or lost.

5. The Base64 filter: the first cleaning line

Txt mode: the stream is text, and anything can be in it — line breaks, spaces, framing, random garbage. The filter keeps a 256-entry pass/discard table and passes exactly 5 ranges: `A-Z`, `a-z`, `0-9`, `+`, `/`. The `=` character is not in the table — and need not be: packets encode into exactly 100 characters with no padding.

The result: the scanner receives a dense stream of "clean" Base64 characters. In binary mode there is no filter in the chain at all — raw bytes go straight to the scanner.

6. The sliding window: how packets are fished out

The scanner moves a fixed-size window across the stream: 100 characters in txt, 75 bytes in bin. The logic is identical:

```
while enough data:
      show the window to the handler
      handler says "it's a packet!" → emit the packet, jump by the window size
      handler says "garbage"        → shift by 1 byte
```

What happens inside the window handler is sections 7–8. Important details:

- **The step-1-on-failure rule** is the guarantee: a packet start cannot "slip between windows". Even if packets are glued to garbage, a window will eventually land exactly on a start.
- **The jump-by-window on success** is the fast lane over a clean stream: consecutive packets are read window after window without enumeration.
- The scanner discards nothing: **the entire stream is retained in memory** (why — sections 10–12).

A numeric example (bin mode): the stream = 10 garbage bytes + a header (75) + sectors. Windows at positions 0–9 are garbage (10 single-byte shifts); position 10 — the header is recognized → packet, jump by 75; then sectors back to back. Total: 10 failures + a streak of successes.

7. How a header is recognized

The window handler gets 75 candidate bytes and decides: packet or not. For a header the check is autonomous — no knowledge about files is needed:

```
first 51 bytes → SHA-256 → do the first 24 bytes equal the packet's last 24 bytes?
```

A match means a correct header (random-coincidence probability 2^{-192}). The accumulator parses the fields (name, size, the file's SHA-256, the ECC volume count) and creates a reception slot. If such a header is already known — it simply increments the header copy counter.

8. How a sector is recognized (and why a header is unavoidable)

A sector is built so that its `D3` hash is computed with the `H5` seed of **a specific file**. Therefore a sector can only be verified by trying the known slots:

```
does the sector number from D1 fall into this slot's 0..T-1 range?
  does D3 == Trunc9(SHA-256(slot's H5 ‖ first 66 bytes of the candidate))?
```

Both checks pass — the sector is "ours"; its payload goes to the slot as a version (with a confirmation counter — see `AssemblyGuide.Tutorial.en.md`, section 4). Hence the fundamental consequence: **until at least one header is recognized, sectors are not recognized at all**. What if the stream started mid-file and a sector arrived before any header? The answer is section 11.

9. The accumulator: from bytes to packets and slots

`StreamProcessor` receives bytes from the scanner in chunks of arbitrary size. Chunk boundaries need not align with packet boundaries, so the accumulator keeps a 75-byte assembly buffer `_pending`: it keeps adding bytes until a full packet is assembled — and only then processes it whole. Packets are never cut by delivery boundaries.
An accepted packet goes down one of two tracks: header → slot (section 7), sector → into **every** slot it fits (section 8). Accepted packets are forwarded down the pipeline in indivisible portions.

10. Reception slots and sector versions

Inside a slot is a dictionary "sector number → version list", sorted by descending confirmation count. A repeat of the same payload reinforces the version; a different payload creates a competitor. Everything that then happens to versions (choice points, exhaustive search, rotation, RS recovery, and the final SHA-256 check) is the subject of the `AssemblyGuide.*` family. It suffices to know here: by assembly time a slot is "all the candidates for every byte of the file + the arbitration".

11. The "sector before header" problem and targeted rebinding

The scenario: the stream was picked up mid-way and the first packet is a data sector. The forward pass does not recognize it (no slots yet) — the window moves on. Is the sector lost forever? No:

1. The scanner **retains the entire stream it has passed** in memory.
2. When a header is later recognized and a slot created, the accumulator raises the "new header accepted" event.
3. The decoder asks **both** scanners (txt and bin) to rebind the retained data: a re-pass runs over the retained bytes in which every window is checked against the new header — "is this a sector belonging to our newcomer?"
4. The found sectors are emitted to the accumulator and settle in the slot.

The rebinding is targeted: only membership in one specific header is checked; the confirmation counters of the other slots are untouched.

12. The rescan: why no jumping

The re-pass has two peculiarities, both important.

Exhaustiveness. The forward pass jumps by the window size after a success — and on a damaged stream that jump can leap over the start of a packet overlapping the recognized one. The re-pass checks the window **at every position**, without jumps: nothing is lost by construction.

The boundary. The re-pass covers not the whole retained stream but only up to the position where the forward pass has already delivered data to the consumer. Beyond that, the forward pass has worked "with knowledge" anyway (the header was already known), and re-emitting would double the confirmations. The boundary is fixed at the moment a batch of data is delivered — carefully, before the handlers run.

13. Finalization: why `Complete` and why it must not be forgotten

At the end of `Scan` the decoder closes the pipeline in order: filter → scanner → accumulator. Each `Complete` means "the input has ended, deliver the remainder". This is not a formality: the scanner's final delivery triggers the rebinding that covers the forward pass's omissions. Anyone relying on reception completeness must wait for `Complete`. After that the progress gets its final 100% (`Done`).

14. Progress and cancellation

Progress is a global 0–100 scale over consumed bytes: `forward-pass position × 100 / input size`. The phase name depends on state: while no header has been found — "header search"; afterwards — "sector search". Cancellation (`CancellationToken`) is checked at every advance and while awaiting pipeline completion; on cancel — an exception, and the slot state remains as it was.

15. File assembly: the entrance to a separate guide

Once the stream has been scanned, the decoder is asked `TryAssemble(header)`: it finds the slot whose header matches byte for byte and invokes the slot's assembly with the RS adapter. From there the mathematics of versions and collisions begins — the subject of `AssemblyGuide.Tutorial.en.md`. The decoder is responsible only for "delivering the cargo to the warehouse"; assembly is a separate shop.

16. The I/O module: sources, sinks, the error channel

Alongside the pipeline live helper classes:

Class	Role
`ByteArraySource`, `StreamSource`, `FileSource`	sources: array, stream (not closed), file
`StreamDataWriter`, `FileDataWriter`, `ByteListWriter`, `PreallocatedBufferWriter`	sinks for the assembly result
`IDataSource.Error`	the error channel: the source's exception; `Completion` ends with it

The point of the error channel: a read failure (say, the disk drops out mid-file) does not stay silent and is not swallowed — it reaches the caller via `Error` / `Completion`, and the user sees an honest "I/O error" instead of an empty result.

17. End-to-end example: a damaged txt stream

Take the stream from `EncoderGuide.Tutorial.en.md` (a 1000-byte file → `H D0…D8 H D9…D17 H`, 20 lines of 100 characters) and damage it: in line 5 replace one character with another Base64-alphabet character, and prepend 37 bytes of garbage (the `>[…]` framing line) before the first line.

What happens:

1. The filter throws out everything outside the alphabet: the decorative line, line breaks, garbage letters outside the alphabet. Only Base64 characters remain.
2. The window lands at the start of the clean stream: `H` — 100 characters decode into 75 bytes, `H5` matches → a header packet, a slot is created, jump by 100.
3. Lines 2–5a: sectors `D0…` are recognized by `D3` with the `H5` seed, jumps of 100.
4. Line 5 (the damaged one): 100 characters decode into 75 bytes, but `D3` fails → shift by 1. The window is now "tail of line 5 + head of line 6" — it decodes but is not recognized → another shift… about 99 failed windows until the window lands on the start of line 6.
5. Lines 6–21: clean jumps of 100 again, including the middle and final headers (the header copy counter grows to 3).
6. `Complete`: the remainders are delivered, the final rebinding picks up what was missed.
7. The result: exactly one sector is lost (#4 — line 5); at 10% ECC (`M = 2`) it will be reconstructed by Reed–Solomon during assembly. The file assembles bit for bit.

The price of one spoiled character is ~99 extra window checks: a trifle compared to losing a packet.

18. What is guaranteed

1. A packet is recognized only on an exact hash match: garbage and forgeries do not enter the slots (2^{-72} per sector, 2^{-192} per header).
2. No correct packet is lost because of surrounding garbage: the step-1 rule on failure and the exhaustive re-pass guarantee it.
3. A sector that arrived before its header is picked up by the rebinding.
4. Confirmations are not doubled: the re-pass is bounded by the delivery boundary.
5. An I/O failure reaches the user via `Error` rather than being masked.

19. Self-check questions

1. Why does the filter not pass `=`, and why is that okay? (section 5)
2. What happens on window-recognition failure, and why is the step exactly 1? (section 6)
3. Why is the header verified autonomously but a sector is not? (sections 7–8)
4. Why does the scanner retain the whole stream? (sections 11–12)
5. Why does the re-pass not jump by the window size? (section 12)
6. What happens if you forget to call `Complete`? (section 13)
7. How many failed windows does one spoiled Base64 character cost? (section 17)
8. How is a sector that arrived before the header handled? (section 11)
9. Where does a sector that fits two slots at once go? (section 9)
10. How does the user learn about a disk failure mid-scan? (section 16)